> **Topic:** Pointers, Heap & Stack, Dynamic Allocation, RAII, `const`, `static`, and Smart Pointers.
> **Source:** CSC3002 Lecture 7 — Dynamic Memory Management, Prof. Ruizhi Chen. 7-CSC3002 Dynamic memory manage

---

⭐ Key Takeaways (TL;DR)

1. **Stack vs. Heap**
   - **Stack:** Memory is managed automatically. Local variables disappear when their scope ends.
   - **Heap:** Memory can be allocated dynamically using `new`. Raw allocations must be released using `delete`.
   - **Static storage:** Variables can exist throughout the program's lifetime.
2. **Dynamic Memory Allocation**
   - `new` → Allocate memory.
   - `delete` → Free a single object.
   - `new[]` → Allocate an array.
   - `delete[]` → Free an array.
   - Always match `new` with `delete`, and `new[]` with `delete[]`.
3. **Common Memory Problems**
   - **Memory leak:** Forgetting to free allocated memory.
   - **Dangling pointer:** A pointer refers to memory that has already been freed.
   - **Double free:** Attempting to free the same memory twice.
4. **RAII (Resource Acquisition Is Initialization)**
   - Acquire resources in a constructor.
   - Release resources in a destructor.
   - Resources are automatically cleaned up when the owning object is destroyed.
5. **`const` and `static`**
   - `const` prevents unintended modifications.
   - `static` can preserve values between function calls or share members across class instances.
6. **Smart Pointers**
   - `unique_ptr` → Exclusive ownership.
   - `shared_ptr` → Shared ownership with reference counting.
   - `weak_ptr` → Non-owning reference to an object managed by `shared_ptr`.
   - Smart pointers automate memory management and reduce errors.

---

1. Memory Organization: Stack, Heap, and Static Data

C++ programs use different memory regions for different purposes.

Memory Region	Purpose	Lifetime	Management
Stack	Local variables, function calls	Until scope/function ends	Automatic
Heap	Dynamically allocated objects	Until explicitly released	Manual / Smart pointers
Static Data	Global and static variables	Usually entire program	Automatic

1.1 Stack Memory

Stack memory stores local variables and function call information.

```
void example() {
    int x = 10;
```

```
    int arr[5] = {1, 2, 3, 4, 5};
}
```

What happens?

1. Function `example()` is called.
2. A stack frame is created.
3. Variables `x` and `arr` are stored.
4. When the function returns, the stack frame is removed.

Key point: No `delete` is required.

1.2 Heap Memory

Heap memory is allocated dynamically during program execution.

```
void example() {
    int* p = new int(10);

    cout << *p;  // 10

    delete p;
}
```

What happens?

1. `new int(10)` allocates an integer on the heap.
2. `p` stores the address of that integer.
3. `*p` accesses the integer.
4. `delete p` releases the allocated memory.

Important distinction:

```
int* p = new int(10);
```

- `p` is a local pointer variable (typically stored on the stack).
- `*p` refers to the integer stored on the heap.

```
STACK                       HEAP

p ──────────────────────►  10
(pointer)                   (integer)
```

Deleting `p` releases the heap allocation, **not the pointer variable itself**.

1.3 Why Use Pointers?

Pointers have three main purposes:

1. Efficiently reference large data structures

Instead of copying an entire object, pass its address.

```
void modify(int* p) {
    *p = 100;
}

int x = 10;
modify(&x);

// x = 100
```

2. Dynamic allocation

```
int* p = new int(5);
delete p;
```

3. Linked data structures

Pointers can connect different objects.

```
struct Node {
    int value;
    Node* next;
};
```

This is the foundation of linked lists.

---

2. Dynamic Memory Allocation: `new` and `delete`

2.1 Allocating a Single Object

Syntax:

```
Type* pointer = new Type;
```

Example:

```
int* p = new int(42);

cout << *p;  // 42

delete p;
p = nullptr;
```

Why set `p = nullptr`?

After calling `delete`, the pointer still contains the old address.

```
int* p = new int(10);

delete p;

// p still contains the old address

cout << *p;  // Undefined behavior!
```

This is called a **dangling pointer**.

Setting the pointer to `nullptr` prevents accidentally accessing the deleted memory through that pointer.

However, other pointers to the same allocation are not automatically updated.

2.2 Allocating Dynamic Arrays

Syntax:

```
Type* arr = new Type[size];
```

Example:

```
int* arr = new int[5];

for (int i = 0; i < 5; i++) {
    arr[i] = i;
}

delete[] arr;
```

Critical Rule:

Allocation	Deallocation
`new int`	`delete p`
`new int[10]`	`delete[] p`
`new MyClass`	`delete p`
`new MyClass[10]`	`delete[] p`

Even if the array contains **only one element**, you must use `delete[]`.

```
int* p = new int[1];

delete[] p;  // Correct
delete p;    // Incorrect
```

Example: Returning a Dynamic Array

```
int* createIndexArray(int n) {
    int* arr = new int[n];

    for (int i = 0; i < n; i++) {
        arr[i] = i;
    }

    return arr;
}

int main() {
    int* digits = createIndexArray(10);

    // digits = [0,1,2,3,4,5,6,7,8,9]

    delete[] digits;
}
```

Why use the heap?

The dynamic array continues to exist after `createIndexArray()` returns.

Unlike the function's local variables, its heap allocation is not automatically destroyed when the function ends.

Remember: The caller is responsible for freeing this allocation.

---

3. Dynamic 2D Arrays

There are two implementations covered in the lecture.

3.1 Method 1: Pointer to Pointers (`int**`)

```
int rows = 3;
int cols = 4;

// Allocate an array of row pointers
int** matrix = new int*[rows];

// Allocate each row
for (int i = 0; i < rows; i++) {
    matrix[i] = new int[cols];
}

// Access elements
matrix[1][2] = 100;
```

Memory structure:

```
matrix
  |
  v
+---------+
| row[0]  | ----> [0][0] [0][1] [0][2] [0][3]
| row[1]  | ----> [1][0] [1][1] [1][2] [1][3]
| row[2]  | ----> [2][0] [2][1] [2][2] [2][3]
+---------+
```

Freeing Memory

Delete each row first, then the array of pointers.

```
for (int i = 0; i < rows; i++) {
    delete[] matrix[i];
}

delete[] matrix;
```

Why?

Each row was allocated separately using `new[]`.

Therefore, each row must be deleted separately.

3.2 Method 2: One Continuous Array

A simpler implementation allocates all elements together.

```
int rows = 3;
int cols = 4;

int* matrix = new int[rows * cols];

// Access element (i, j)
matrix[i * cols + j] = 100;

delete[] matrix;
```

Index Formula:



Example:

For a 3×4 matrix, accessing element `(2, 1)`:

So:

```
matrix[9]
```

corresponds to:

```
matrix[2][1]
```

in a conventional 2D representation.

Comparison

Feature	`int**`	Continuous `int*`
Allocation	Multiple `new[]`	One `new[]`
Access	`matrix[i][j]`	`matrix[i * cols + j]`
Deallocation	Delete every row, then row pointers	One `delete[]`
Memory layout	Rows allocated separately	One continuous block

Key takeaway: A continuous allocation simplifies memory management.

---

4. Memory Leaks, Dangling Pointers, and Double Free

These are three common memory management bugs.

4.1 Memory Leak

A **memory leak** occurs when dynamically allocated memory is no longer accessible but has not been freed.

Example:

```
void example() {
    int* p = new int(10);
}
```

Problem:

- Heap memory is allocated.
- The function returns.
- Pointer `p` disappears.
- The allocated integer remains on the heap.
- The program has lost the ability to free it.

Solution:

```
void example() {
    int* p = new int(10);

    delete p;
}
```

4.2 Dangling Pointer

A dangling pointer refers to an object that no longer exists.

```
int* p = new int(10);

delete p;

cout << *p;  // Undefined behavior
```

Solution:

```
delete p;
p = nullptr;
```

However, this only fixes `p`, not other pointers that might reference the same memory.

4.3 Double Free

Double free occurs when the same allocation is released more than once.

```
int* p1 = new int(10);

int* p2 = p1;

delete p1;
delete p2;  // Double free!
```

Why?

Both pointers refer to the same memory.

```
p1 ----+
       |
       v
       10
       ^
       |
p2 ----+
```

After `delete p1`, the memory is already freed.

Calling `delete p2` attempts to free it again.

Key rule: One allocation must not be deleted twice.

---

5. Constructors, Destructors, and RAII

5.1 Destructors

A destructor is a special class function that runs when an object is destroyed.

Syntax:

```
~ClassName();
```

Example:

```
class Example {
public:
    Example() {
        cout << "Created\n";
    }
```

```
    ~Example() {
        cout << "Destroyed\n";
    }
};

int main() {
    Example obj;
}
```

Output:

```
Created
Destroyed
```

Important:

- Stack objects are automatically destroyed when their scope ends.
- Heap objects created using `new` must be destroyed using `delete`.
- Calling `delete` on an object also calls its destructor.

5.2 RAII — Resource Acquisition Is Initialization

Definition:

> Resource management is tied to object lifetime.

In simpler words:

- **Constructor:** Acquire resources.
- **Destructor:** Release resources.

Example:

```
class DynamicArray {
private:
    int* data;

public:
    DynamicArray(int size) {
        data = new int[size];
    }

    ~DynamicArray() {
        delete[] data;
    }
};

int main() {
    DynamicArray arr(10);
}
```

Execution sequence:

```
1. Create arr
       |
       v
2. Constructor called
       |
       v
3. Allocate heap memory
       |
       v
4. arr goes out of scope
```

```
       |
       v
5. Destructor called
       |
       v
6. Heap memory freed
```

Why is RAII useful?

Without RAII, programmers must manually track every allocation.

With RAII, objects automatically release the resources they own.

Important limitation: This simple class demonstrates RAII but does not implement safe copying. If an owning object is copied, both copies may point to the same allocation, causing double deletion. In production code, copying must be handled correctly, or ownership should be managed using standard containers or smart pointers.

---

6. Heap-Stack Diagrams and Function Calls

Heap-stack diagrams illustrate where variables and dynamically allocated objects exist.

Consider:

```
struct Point {
    int x;
    int y;
};

void modify(int* arr, Point pt, double& total) {
    Point* p = new Point;

    arr[1] = pt.x;
    total += pt.y;

    delete p;
}

int main() {
    double total = 0.0;

    Point pt{1, 2};

    int* arr = new int[5];

    modify(arr, pt, total);

    delete[] arr;
}
```

The three parameters behave differently:

Parameter	Passing Method	Behavior
`int* arr`	Pointer	Can modify the original array
`Point pt`	Value	Copies the Point object
`double& total`	Reference	Can modify the original variable

After calling `modify()`:

```
arr[1] == 1
```

```
total == 2.0
```

Important Concepts

Pass by Value

```
void f(int x)
```

- Receives a copy.
- Changes do not affect the original.

Pass by Pointer

```
void f(int* p)
```

- Receives a copy of an address.
- Can modify the object through the address.

Pass by Reference

```
void f(int& x)
```

- Refers to the original variable.
- Changes affect the original.

Stack vs. Heap During Function Calls

- Function calls create new stack frames.
- Returning from a function removes its stack frame.
- Heap allocations remain until released.
- Losing the only pointer to a heap allocation causes a memory leak.

---

7. Garbage Collection vs. C++ Memory Management

Languages such as Python and Java use garbage collection to reclaim unreachable objects automatically.

Traditional C++ does not have an automatic garbage collector as part of its standard memory management model.

Feature	Python / Java	C++
Garbage collection	Yes	Not built into standard C++
Explicit memory control	Generally limited	Yes
Manual `delete`	No	Required for owning raw `new` allocations
Automatic resource management	Garbage collection / other mechanisms	RAII and smart pointers

Advantages of Garbage Collection

- Easier memory management.
- Reduces many memory leaks.
- Reduces dangling-pointer and double-free errors.

Disadvantages

- Additional resource consumption.
- Potential performance overhead.
- Possible pauses during collection.

C++ approach: Use RAII and smart pointers to obtain automatic cleanup without requiring a garbage collector.

C-Style Dynamic Memory Allocation

The lecture briefly mentions:

Function	Purpose
`malloc()`	Allocate memory
`calloc()`	Allocate and zero-initialize memory
`realloc()`	Resize an allocated block
`free()`	Release allocated memory

Exam note: The professor explicitly states that memorizing these C functions is not required.

---

8. The `const` Keyword

The `const` keyword prevents modification through a specified variable, reference, or function interface.

The lecture introduces three major uses.

8.1 Constant Variables

```
const int MAX_SIZE = 100;

// MAX_SIZE = 200;  // Error
```

Once initialized, the variable cannot be reassigned.

8.2 Constant Reference Parameters

```
void printValue(const int& x) {
    cout << x;
}
```

Advantages:

- Avoids unnecessary copying.
- Prevents modification through the reference.

For example:

```
void printName(const string& name) {
    cout << name;

    // name = "New Name";  // Error
}
```

This is especially useful for large objects.

8.3 Constant Member Functions

A `const` member function promises not to modify the object's ordinary data members.

```
class Student {
private:
    int age;

public:
```

```
    Student(int a) : age(a) {}

    int getAge() const {
        return age;
    }

    void setAge(int a) {
        age = a;
    }
};
```

Notice:

```
int getAge() const
```

The `const` appears **after the parameter list**.

This means the function cannot modify ordinary member variables.

For example:

```
int getAge() const {
    age = 20;  // Error
    return age;
}
```

Constant Data Members

Constant members can be initialized using a constructor initializer list.

```
class A {
public:
    const int value;

    A(int x) : value(x) {}
};
```

After initialization, `value` cannot be changed.

Summary of `const`

Syntax	Meaning
`const int x`	Constant variable
`const T& x`	Read-only reference
`int get() const`	Const member function
`const int value;`	Constant data member

Key takeaway: Use `const` to prevent unintended changes and improve code safety.

---

9. The `static` Keyword

The meaning of `static` depends on where it is used.

9.1 Static Local Variables

A static local variable retains its value between function calls.

```
void counter() {
    static int count = 0;

    count++;

    cout << count << endl;
}

int main() {
    counter();
    counter();
    counter();
}
```

Output:

```
1
2
3
```

Why?

Normally, local variables are recreated when a function is called.

A `static` local variable is initialized only once and preserves its value.

Compare:

```
void counter() {
    int count = 0;
    count++;
    cout << count << endl;
}
```

Output:

```
1
1
1
```

9.2 Static Class Members

Static class variables are **shared by all objects of the same class**.

```
class Student {
public:
    static int count;

    Student() {
        count++;
    }
};

int Student::count = 0;

int main() {
    Student a;
    Student b;
    Student c;

    cout << Student::count;
}
```

Output:

```
3
```

There is only one `Student::count`, regardless of how many `Student` objects exist.

9.3 Static Member Functions

Static methods belong to the class rather than individual objects.

```
class Example {
public:
    static int value;

    static int getValue() {
        return value;
    }
};

int Example::value = 10;
```

Calling:

```
cout << Example::getValue();
```

Output:

```
10
```

Important rule:

A static member function cannot directly access non-static data members.

```
class A {
public:
    int x = 10;
    static int y;

    static void print() {
        cout << y;  // OK
        // cout << x;  // Error
    }
};
```

This is because static functions do not belong to any particular object.

9.4 Static Global Variables and Functions

At file scope, `static` can restrict a variable or function to the current source file.

```
static int counter = 0;

static void helper() {
    counter++;
}
```

These names cannot be directly accessed from another source file through external linkage.

Summary of `static`

Usage	Meaning
Static local variable	Retains value between function calls
Static global variable	Restricted to the current source file
Static global function	Restricted to the current source file
Static class variable	Shared by all class instances
Static class function	Can directly access only static class members

---

10. Smart Pointers

Smart pointers automatically manage dynamically allocated objects.

They are provided by:

```
#include <memory>
```

Three important types:

Smart Pointer	Ownership	Main Purpose
`unique_ptr`	Exclusive	One owner
`shared_ptr`	Shared	Multiple owners
`weak_ptr`	Non-owning	Observe shared objects

10.1 `unique_ptr` — Exclusive Ownership

A `unique_ptr` owns an object exclusively.

When the pointer goes out of scope, its managed object is automatically destroyed.

Example:

```
#include <memory>
using namespace std;

int main() {
    auto p = make_unique<int>(10);

    cout << *p;  // 10
}
```

No manual `delete` is needed.

Dynamic Array Example

```
auto arr = make_unique<int[]>(5);

for (int i = 0; i < 5; i++) {
    arr[i] = i;
}

cout << arr[3];  // 3
```

The array is automatically freed.

Important Property

A `unique_ptr` cannot be copied because ownership is exclusive.

```
auto p1 = make_unique<int>(10);

// auto p2 = p1;  // Error
```

Ownership can instead be transferred using move semantics:

```
auto p2 = std::move(p1);
```

After the move:

- `p2` owns the integer.
- `p1` no longer owns it.

10.2 `shared_ptr` — Shared Ownership

A `shared_ptr` allows multiple pointers to own the same object.

It uses **reference counting** to track the number of owners.

Example:

```
#include <memory>
using namespace std;

int main() {
    auto p1 = make_shared<int>(100);

    cout << p1.use_count() << endl;  // 1

    {
        auto p2 = p1;

        cout << p1.use_count() << endl;  // 2
    }

    cout << p1.use_count() << endl;  // 1
}
```

How reference counting works:

```
Create p1
    |
    v
count = 1
    |
    v
Copy p1 to p2
    |
    v
count = 2
    |
    v
p2 goes out of scope
    |
    v
count = 1
    |
```

```
    v
p1 goes out of scope
    |
    v
count = 0
    |
    v
Object destroyed
```

Key rule:

[\boxed{\text{Reference count} = 0 \Rightarrow \text{Destroy managed object}}]

Unlike `unique_ptr`, a `shared_ptr` can be copied.

10.3 `weak_ptr` — Non-Owning Reference

A `weak_ptr` observes an object managed by `shared_ptr` without owning it.

Important properties:

- Does not increase the shared ownership count.
- Does not extend the object's lifetime.
- Must be converted to a temporary `shared_ptr` using `.lock()` before accessing the object.

Example:

```
#include <memory>
using namespace std;

int main() {
    auto sp = make_shared<int>(100);

    weak_ptr<int> wp = sp;

    cout << sp.use_count() << endl;  // 1

    if (auto p = wp.lock()) {
        cout << *p << endl;  // 100
    }
}
```

Why Use `.lock()`?

The original object might already have been destroyed.

```
weak_ptr<int> wp;

{
    auto sp = make_shared<int>(100);

    wp = sp;
}

// sp destroyed; managed object also destroyed

if (auto p = wp.lock()) {
    cout << *p;
} else {
    cout << "Object no longer exists";
}
```

Output:

```
Object no longer exists
```

Important distinction: A successful `.lock()` creates a temporary `shared_ptr`, which increases the shared ownership count while it exists.

---

11. Comparing the Three Smart Pointers

Consider an object stored on the heap:

```
make_shared<int>(100);
```

`unique_ptr`

```
unique_ptr
    |
    v
  [100]

Only one owner
```

- Exclusive ownership.
- Cannot be copied.
- Automatically deletes its object.

`shared_ptr`

```
shared_ptr A ----+
                 |
                 v
               [100]
                 ^
                 |
shared_ptr B ----+

Reference count = 2
```

- Multiple owners.
- Reference counting.
- The object is destroyed when the last owner disappears.

`weak_ptr`

```
shared_ptr ----+
               |
               v
             [100]
               ^
               |
weak_ptr ------+

Reference count = 1
```

- Observes the object.
- Does not contribute to ownership.
- Must use `.lock()` to access it safely.

Which One Should I Use?

Situation	Recommended Pointer
Only one object owns a resource	`unique_ptr`
Multiple objects must share ownership	`shared_ptr`
Need to observe a shared object without owning it	`weak_ptr`

General rule: Prefer `unique_ptr` unless shared ownership is necessary.

---

12. Common Exam Questions & Mistakes

Q1. What is the difference between stack and heap?

Answer:

Stack memory is managed automatically according to scope and function lifetime.

Heap memory supports dynamic allocation and requires appropriate memory management.

Q2. What happens if we forget `delete`?

A memory leak may occur.

```
int* p = new int(10);

// Missing delete
```

Q3. What is wrong with this code?

```
int* arr = new int[10];

delete arr;
```

Answer: Incorrect deallocation.

It should be:

```
delete[] arr;
```

Q4. What happens after `delete p`?

The allocated object is destroyed, but the pointer variable itself remains alive if its scope has not ended.

It becomes dangling.

Q5. When is a destructor called?

- When a local object goes out of scope.
- When a dynamically allocated object is deleted.
- When an owning smart pointer destroys its managed object.

Q6. What does RAII mean?

Resource Acquisition Is Initialization.

Acquire resources during object initialization and release them during destruction.

Q7. What is the difference between `const` and `static`?

- `const` : Restricts modification.
- `static` : Changes storage duration, linkage, or class-level sharing depending on context.

Q8. Can a static function access non-static members?

Not directly.

Static functions have no implicit object instance.

Q9. What happens when a `shared_ptr` is copied?

The shared ownership reference count increases by one.

Q10. Does `weak_ptr` increase the ownership count?

No.

However, successfully calling `.lock()` creates a temporary `shared_ptr` , which contributes to the reference count.

⭐ Final Key Takeaways — Quick Revision

Memory Management

Concept	Remember
Stack	Automatic storage management
Heap	Dynamic allocation
`new`	Allocate one object
`delete`	Free one object
`new[]`	Allocate an array
`delete[]`	Free an array
Memory leak	Allocated memory is not released and becomes unreachable
Dangling pointer	Pointer refers to an object that no longer exists
Double free	Same allocation is freed twice
Destructor	Performs cleanup when an object is destroyed
RAII	Ties resource lifetime to object lifetime

`const` and `static`

Keyword	Core Meaning
`const` variable	Cannot be reassigned
`const T&`	Prevents modification through the reference
`function() const`	Const member function
Static local variable	Retains value between calls
Static class variable	Shared across instances
Static class function	No direct access to non-static members

Smart Pointers

Pointer	Ownership	When Object Is Deleted
`unique_ptr`	Exclusive	Owner releases or goes out of scope
`shared_ptr`	Shared	Last owner releases ownership

Pointer	Ownership	When Object Is Deleted
`weak_ptr`	None	Does not determine lifetime

Five Rules to Memorize

Rule 1 — Match Allocation and Deallocation

```
new       → delete
new[]     → delete[]
```

Rule 2 — Never Access Freed Memory

```
delete p;
p = nullptr;
```

Rule 3 — Understand Object Lifetime

```
Stack object → Automatically destroyed
Heap object  → Requires ownership management
```

Rule 4 — Prefer RAII

```
Constructor → Acquire resources
Destructor  → Release resources
```

Rule 5 — Prefer Smart Pointers for Dynamic Ownership

```
unique_ptr → One owner
shared_ptr → Multiple owners
weak_ptr   → Non-owning observer
```

The Most Important Idea of This Lecture

> **Dynamic memory management is fundamentally about resource ownership and lifetime.**
>
> Understand **who owns the memory**, **how long it remains valid**, and **who is responsible for releasing it**.
>
> `new` and `delete` provide manual control, RAII ties cleanup to object lifetime, and smart pointers make ownership management safer and easier.